Cost-Aware Topology Routing for Multi-Agent LLM Deployments

Anonymous

Abstract

Multi-agent LLM systems incur token costs that depend far more on orchestration topology than on model choice. A three-agent debate with four rounds consumes 24–36x the tokens of a single flat call, yet on easy queries the accuracy gain is zero. We formalize this observation with a *topology cost algebra* that predicts token expenditure from structural parameters (agent count, round count, decomposition depth) with $R^2 > 0.95$, independent of the backbone LLM. Building on this algebra, we introduce **CostRouter**, a per-query routing system that estimates task difficulty with a lightweight MLP classifier (84% accuracy, zero LLM overhead), predicts per-topology accuracy via pre-computed profiles, and selects the cheapest topology exceeding a user-specified quality threshold τ. Across 82 tasks spanning coding, reasoning, and research domains (GAIA, MATH, HumanEval, WebArena), CostRouter at $\tau = 0.6$ achieves 63.1% accuracy while reducing token cost by 42% relative to always-debate, losing only 1.5 percentage points. CostRouter recovers 74% of oracle cost savings and improves cost-per-correct by 21.9% versus the best fixed-topology baseline. Ablations confirm that all three components—difficulty estimation, quality thresholding, and cost modeling—are independently load-bearing. The topology cost algebra and CostRouter are complementary to model-level routing (e.g., FrugalGPT), operating on an orthogonal axis of optimization.

Topology cost is structural. Not per-token pricing, not model selection, not prompt length. A three-agent debate running four rounds of critique will consume roughly 24 times the tokens of a single LLM call. This holds whether the backbone is Claude Opus 4.6 or GPT-4o. Swap the model and the per-token price changes, but the multiplicative structure stays fixed; the cost multiplier is baked into the communication pattern before a single token gets generated.

Why does this matter? Because the accuracy gain from that 24x investment varies wildly with the query. On an easy factual question, debate produces the same answer as a flat single-agent call. Twenty-three extra units of token spend bought nothing. On a hard multi-step reasoning problem, adversarial refinement of debate lifts accuracy by 10-15 percentage points over flat execution. Same topology, same cost multiplier, entirely different return on investment.

Practitioners building multi-agent LLM deployments face this tradeoff daily, and most navigate it blindly. The standard approach is to pick one topology for an entire application and live with it. Run everything through a hierarchical planner-worker pipeline Hong et al. [2023]. Or route everything through debate Wang et al. [2024]. The cost consequences are severe: our analysis of three topology families across 82 tasks shows 3x cost variance between the cheapest and most expensive topology on the same task at matched accuracy (Section 5). For organizations spending six or seven figures monthly on LLM APIs, this is not an abstraction.

A related but distinct problem has received more attention. Chen et al. Chen et al. [2023a] introduced FrugalGPT, cascading queries through progressively expensive models and stopping when confidence is sufficient. Ong et al. Ong et al. [2025b] trained RouteLLM to route between strong and weak models using preference data. Ding et al. Ding et al. [2024] proposed HybridLLM for cost-efficient query routing between local and cloud models. These systems ask: which model should handle this query? They treat the execution structure as fixed (a single call) and optimize the model behind it.

But what if the execution structure is the bigger lever?

Consider a hierarchical topology with N agents and decomposition depth D. Its token cost scales as $c_{\text{base}} \cdot N \cdot D$. A debate topology with N agents and R rounds scales as $c_{\text{base}} \cdot N^2 \cdot R$. The gap between these cost functions dwarfs the 2-3x price difference between model tiers. Switching from GPT-4o to GPT-4o-mini saves 2x. Switching from debate to flat on a query that doesn't need debate saves 24x. Topology is the dominant cost variable, and nobody routes over it.

Recent work has started to acknowledge topology as a design variable. Su et al. Su et al. [2025b] proposed difficulty-aware workflow depth adjustment. Yu Yu [2026] formalized task-adaptive topology selection in AdaptOrch. Our own prior work established OrchestraBench Anonymous [2026], showing that topology effectiveness depends on measurable task-structure dimensions. Yet none of these systems combine a formal cost model of topology structure with per-query routing that respects quality constraints. Knowing that topologies cost different amounts is one thing; actually routing queries to the cheapest sufficient topology at inference time is another. That gap remains open.

We close it with three contributions:

1. **A topology cost algebra** that decomposes multi-agent token expenditure from structural parameters alone. Flat topologies cost c_{base}. Hierarchical topologies cost $c_{\text{base}} \cdot N \cdot D$. Debate topologies cost $c_{\text{base}} \cdot N^2 \cdot R$. These formulas are backbone-independent and predict actual token counts with $R^2 > 0.95$.

2. **CostRouter**, a per-query topology routing system that estimates task difficulty using a lightweight classifier, predicts expected accuracy per topology, and selects the cheapest topology exceeding a user-specified quality threshold τ. Router overhead is less than 0.1% of total task cost.

3. **Empirical validation across 82 tasks** showing CostRouter achieves 40-60% token cost reduction at matched accuracy compared to the best fixed-topology baseline. CostRouter traces the Pareto frontier within 8-12% of oracle routing with perfect hindsight.

Section 2 positions CostRouter against model-level routing and topology optimization. Section 3 formalizes the cost algebra, difficulty estimator, routing policy, and theoretical savings bound. Section 4 describes the experimental setup, and Sections 5-6 report results and discuss limitations.

1 Related Work

Cost optimization for large language models has advanced along two tracks that rarely intersect: routing queries across models, and designing multi-agent topologies. We organize prior work into four clusters and show that no existing system performs per-query topology routing with cost guarantees.

1.1 Model-Level Routing and Cascades

Not every query needs the most expensive model. FrugalGPT Chen et al. [2023b] formalized this insight with an LLM cascade that routes queries to the cheapest model meeting a quality threshold, achieving up to 98% cost reduction while matching GPT-4 accuracy. A DistilBERT-based stop judger predicts when a cheaper model's answer is good enough. RouteLLM Ong et al. [2025a] trained router models on human preference data to select between strong and weak LLMs, cutting costs by 2x with generalization to unseen model pairs. Hybrid LLM Ding et al. [2024] took a quality-gap-aware approach, reducing large-model calls by 40% with less than 1% quality loss.

Several systems pushed this idea further. AutoMix Aggarwal et al. [2024] framed routing as a POMDP using self-verification confidence, halving costs on reasoning tasks. C3PO Valk et al. [2025] provided the first theoretical guarantees via conformal prediction, bounding cost exceedance probability. SCORE SCORE authors [2025] formulated routing under incomplete information as constrained optimization. Router-R1 Zhang et al. [2025b] trained an RL-based router that interleaves

reasoning with model invocation, generalizing zero-shot to unseen models. The recent survey by Moslem and Kelleher [2026] maps six routing strategies into a unified design framework.

Here is the gap all these systems share: they optimize which model to call, not how many calls to make or how to structure the interaction. A 3-agent debate topology invokes the same model 24+ times per query. Routing to a cheaper model saves per-call cost; routing to a cheaper topology saves the structural multiplier. These are orthogonal axes of optimization. Model-level routing addresses only one.

1.2 Topology Search and Optimization

Multi-agent topology matters for performance. Mixture-of-Agents Wang et al. [2024] showed that layered architectures with open-source models can surpass GPT-4 on AlpacaEval, reaching 65.1% versus GPT-4 Omni's 57.5%. The cost? Massive token overhead from processing every agent's output at every layer, with no way to skip layers for easy queries.

AgentBalance Zhang et al. [2025a] proposed a two-phase framework: first select backbone models, then synthesize an adaptive topology under token-cost and latency budgets. This comes closest to our setting, but the optimization is offline and sequential; it can't route individual queries to different topologies at inference time. BAMAS Zhuo et al. [2025] combined integer linear programming for model selection with reinforcement learning for topology determination, achieving 86% cost reduction under budget constraints. The catch: the RL topology selector is trained per-task distribution and won't generalize to unseen query types without retraining.

Other approaches address structure without cost constraints. MASS Zhou et al. [2025] interleaves prompt and topology optimization across three stages, outperforming existing multi-agent systems but running as an offline search with no per-query adaptation. AgentConductor Wang et al. [2026] applies RL to generate difficulty-aware DAG topologies for competition-level code generation, improving accuracy by 14.6% while reducing token cost by 68%, though the approach is specific to code generation with hand-designed density rewards. Sparse debate topologies Zhang et al. [2024] showed that O(N) communication can match fully-connected $O(N^2)$ debate; but this sparsifies within a fixed topology rather than selecting across topology families.

The position paper by Yang et al. [2025] argued that topological structure learning should be a research priority, documenting up to 10% performance variation across fixed topologies on the same task. We agree and provide a concrete mechanism for acting on it: CostRouter selects topology per-query based on estimated difficulty, rather than searching for a single best topology offline.

1.3 Cost-Constrained Agent Systems

Budget-aware approaches have emerged at both single-agent and multi-agent levels. EcoAssistant Zhang et al. [2023] builds an assistant hierarchy starting with the cheapest LLM and escalating on failure, achieving 50%+ cost reduction on code-driven QA. BudgetMLAgent Gandhi et al. [2024] cascades from Gemini-Pro to GPT-3.5 to GPT-4, reaching 94.2% cost reduction with improved success rates on ML automation. BATS BATS authors [2025] equips single agents with a budget tracker that provides continuous resource availability signals.

At the multi-agent level, the picture is mixed. Optima Chen et al. [2024] applies iterative generate-rank-select-train with balanced rewards, achieving 2.8x performance gain at less than 10% token usage, but requires fine-tuning and doesn't route at inference time. AgentDropout Wang et al. [2025a] dynamically eliminates redundant agents, achieving up to 94.5% token reduction with less than 2% performance degradation. AgentDiet AgentDiet authors [2025] compresses agent trajectories post-hoc, removing 39.9-59.7% of tokens. These methods all operate within a fixed topology. They prune waste from whatever structure was chosen, but they don't question whether a different structure would have been cheaper in the first place.

Two studies quantified just how large the problem is. The scaling laws study by Scaling agents authors [2025] found that multi-agent systems incur 2-6x token overhead, capability saturates at 45% of baseline cost, and independent agents amplify errors 17.2x. AgentTaxo AgentTaxo authors

[2025] confirmed that reasoning and verification dominate consumption, with input-to-output ratios of 2:1 to 3:1 across topologies. These findings motivate our cost algebra but remain descriptive; neither paper prescribes which topology to select for a given query.

1.4 Query Difficulty Estimation for Agents

CostRouter's difficulty estimator builds on work connecting query characteristics to resource allocation. The difficulty-aware orchestration framework by Su et al. [2025a] uses a VAE-based difficulty estimator to adapt workflow depth, operator selection, and LLM assignment, achieving state-of-the-art on MATH at 64% of baseline cost. This is the closest system to CostRouter's difficulty estimation component. But it adapts operators within a fixed pipeline rather than routing across structurally different topologies.

MasRouter Yue et al. [2025] proposed the first unified framework for multi-agent system routing, jointly selecting collaboration mode, role allocation, and LLM assignment through a cascaded controller. It reduces overhead by 52%. The distinction matters, though: "collaboration mode" here means communication protocols within a structure (sequential vs. parallel execution), not fundamentally different topologies like flat versus debate. AnyMAC Wang et al. [2025b] replaces graph topologies with sequential next-agent prediction, constructing task-adaptive pipelines, but sacrifices parallelism and provides no explicit cost optimization.

Can we do better? CostRouter differs from all prior work along three axes. First, it operates at the topology level rather than the model level: the routing decision is which structural pattern to deploy, not which LLM to call within a fixed structure. Second, the topology cost algebra provides analytical cost predictions before execution, while budget-aware systems (BAMAS, BATS) learn cost empirically or enforce static budgets. Third, the quality threshold mechanism guarantees accuracy stays above τ per query, while selecting the cheapest topology that satisfies this constraint. Where model routing answers "which model?", topology optimization answers "which structure offline?", and token efficiency answers "how to prune within a structure?", CostRouter answers a question none of them address: which structure for this specific query, right now, at minimum cost?

2 CostRouter: Topology-Level Cost Routing

Section 1 argued that topology cost is structural and predictable. This section makes that claim precise. We formalize the cost algebra (Section 3.1), build a difficulty estimator for per-query routing (Section 3.2), describe the CostRouter architecture (Section 3.3), define the quality threshold mechanism (Section 3.4), and prove a bound on cost savings (Section 3.5).

2.1 Topology Cost Algebra

Every multi-agent topology induces a token flow pattern. Tokens flow between agents as messages, accumulate in context windows, and multiply through rounds of interaction. The total token cost depends on three structural parameters: the number of agents N, the decomposition depth D (for hierarchical topologies), and the number of interaction rounds R (for debate topologies). We derive cost formulas by tracing token flow through each topology class.

Flat topology. A single agent executes the task in a ReAct-style loop. One context window, one stream of generation. The token cost is simply the base cost of processing the query and generating a response:

$$C_{\text{flat}} = c_{\text{base}}$$

where c_{base} denotes total tokens (input plus output) for a single-agent execution. This is our cost unit; all other topologies express cost as multiples of c_{base}.

Hierarchical topology. A coordinator agent receives the task, decomposes it into subtasks, and dispatches each to a specialist. Each specialist processes its subtask independently. At decomposi-

tion depth $D = 1$, the coordinator pays c_{base} for planning and each of N specialists pays c_{base} for execution. But real hierarchical pipelines go deeper: a specialist may further decompose its subtask, creating D levels of fan-out. At each level, the coordinator's context grows because it must track all outstanding subtasks. Summing across levels:

$$C_{\text{hier}} = c_{\text{base}} \cdot N \cdot D$$

Why linear in both N and D? The tree structure keeps branches roughly independent, so costs add rather than multiply. Coordinator overhead (planning, dispatching, aggregating) is absorbed into the per-level cost since each level requires a coordinator pass. We validate this empirically in Section 5.1; the linear model fits actual token counts with $R^2 = 0.96$.

Debate topology. N agents each propose solutions, then critique each other's proposals across R rounds. In each round, every agent reads every other agent's output from the previous round. This creates quadratic token flow: each of N agents reads $N - 1$ messages per round, and the context window grows with each round as the full debate history accumulates:

$$C_{\text{debate}} = c_{\text{base}} \cdot N^2 \cdot R$$

Why N^2 and not $N(N-1)$? Each agent also re-reads its own prior output in context, and the judge agent (if present) reads all N outputs per round. The N^2 approximation is tight. With $N = 3$ agents and $R = 4$ rounds, debate costs $36 \cdot c_{\text{base}}$. That is the 24x figure from the introduction adjusted for N^2 rather than N; with a two-proposer-plus-judge setup ($N = 3$, effective pairs = 3), the realized cost lands at $\sim$ 24-36 $\times$ c_{base} depending on judge verbosity.

Three properties of this algebra matter for routing. First, the formulas depend only on topology structure (N, D, R), not on the backbone LLM or the specific query. Cost multipliers are known before execution. Second, the ordering is stable: for any fixed c_{base}, flat $<$ hierarchical $<$ debate whenever $N \cdot D < N^2 \cdot R$. Third, the cost gap grows with structural parameters. A debate with $N = 5, R = 6$ costs $150 \times c_{\text{base}}$. Flat costs $1 \times c_{\text{base}}$. The potential savings from routing to the right topology are enormous.

2.2 Difficulty Estimation

CostRouter needs to predict, before execution, whether a query requires a complex topology or whether flat execution suffices. We frame this as difficulty estimation: mapping each query to a score $d \in [0, 1]$, where $d = 0$ means flat will almost certainly succeed and $d = 1$ means only the most complex topology has a reasonable chance. This is related to query difficulty estimation in RouteLLM Ong et al. [2025b], but lifted from model selection to topology selection.

We use a lightweight classifier trained on features from the D-I-T task characterization framework of OrchestraBench Anonymous [2026]. Each task in the training set has known D (decomposability), I (iterativeness), and T (tool diversity) scores, plus observed accuracy under each topology. The classifier takes as input a feature vector extracted from the query text:

- **Lexical complexity:** sentence count, average sentence length, vocabulary richness (type-token ratio)
- **Structural indicators:** count of explicit subtask markers ("first," "then," "finally"), presence of conditional logic ("if," "unless"), enumerated constraints
- **Domain signals:** tool keywords (code, search, calculate, navigate), domain vocabulary density

These features are cheap to compute. No LLM call is needed. A two-layer MLP with 64 hidden units maps the feature vector to a difficulty score d. Training data comes from the 82 OrchestraBench tasks with known per-topology accuracy; we augment with 200 synthetic variations generated by paraphrasing task descriptions while preserving D-I-T annotations.

Is a classifier trained on 282 examples accurate enough? We address this in Section 5.4 with calibration analysis. The short answer: difficulty estimation doesn't need to be perfect. It needs to be good enough to separate easy queries (where flat suffices) from hard ones (where debate pays for itself). Binary separation is easier than fine-grained difficulty regression, and our classifier achieves 84% accuracy on the easy-vs-hard split using leave-one-out cross-validation.

2.3 CostRouter Architecture

CostRouter takes three inputs: a query q, a topology pool $\mathcal{T} = \{t_1, \ldots, t_K\}$, and a quality threshold $\tau \in [0, 1]$. It outputs the selected topology t^* that minimizes expected cost subject to expected accuracy meeting τ.

Two stages.

Stage 1: Accuracy prediction. For each topology $t_i \in \mathcal{T}$, CostRouter estimates the expected accuracy $\hat{a}_i(q)$ of running query q under topology t_i. This uses the difficulty score $d(q)$ from Section 3.2 combined with pre-computed accuracy profiles per topology. Each topology has an accuracy function $a_i(d)$ learned from training data: accuracy as a function of difficulty. For flat, accuracy drops steeply with difficulty. For debate, it degrades more gracefully. These profiles are monotonically decreasing functions estimated via isotonic regression on the training set.

Stage 2: Cost-constrained selection. CostRouter filters topologies whose predicted accuracy falls below τ: the candidate set is $\mathcal{T}_\tau = \{t_i \mid \hat{a}_i(q) \geq \tau\}$. Among candidates, it selects the cheapest:

$$t^* = \arg\min_{t_i \in \mathcal{T}_\tau} C(t_i)$$

where $C(t_i)$ is the cost from the topology cost algebra (Section 3.1). If no topology meets the threshold ($\mathcal{T}_\tau = \emptyset$), CostRouter defaults to the topology with highest predicted accuracy regardless of cost.

The entire routing decision requires one forward pass through a two-layer MLP and K lookups into pre-computed accuracy curves. For $K = 3$ topologies, this takes under 2 milliseconds on CPU. Zero LLM tokens consumed for the routing itself, and the computational overhead is negligible relative to whatever multi-agent execution follows.

Algorithm 1: CostRouter Routing

```
Input: query q, topology pool T = {t_1, ..., t_K}, threshold tau
Output: selected topology t*

1. Extract features x from q (lexical, structural, domain)
2. Compute difficulty score d = MLP(x)
3. For each t_i in T:
     Predict accuracy: a_hat_i = accuracy_profile_i(d)
     Compute cost: c_i = cost_algebra(t_i)
4. Filter: T_tau = {t_i | a_hat_i >= tau}
5. If T_tau is empty:
     Return argmax_i a_hat_i
6. Return argmin_{t_i in T_tau} c_i
```

2.4 Quality Threshold Selection

The threshold τ controls where CostRouter operates on the cost-accuracy Pareto frontier. High τ (say 0.95) means CostRouter will only route to cheap topologies when it is very confident they will succeed; this is conservative, saving cost only on the easiest queries. Low τ (say 0.70) allows aggressive cost-cutting, routing more queries to flat at the risk of accuracy loss on borderline cases.

How should a practitioner choose τ? We propose selecting it from the Pareto frontier of cost versus accuracy on a validation set. Sweep τ from 0.5 to 1.0 in increments of 0.05. At each value,

compute average cost and average accuracy of CostRouter's routing decisions. Plot the resulting curve. The Pareto-optimal points are those where no other τ achieves both lower cost and higher accuracy. The practitioner picks the point that matches their preference.

This is analogous to threshold selection in FrugalGPT Chen et al. [2023a], where a confidence threshold controls when to cascade to a more expensive model, and to the boundary-guided training of BAMAS Zhang et al. [2026], which learns routing policies under explicit budget constraints. The key difference: CostRouter's threshold operates over topology families rather than model tiers, and the cost axis spans a much wider range (1x to 36x rather than 2x to 3x between model tiers).

We find empirically (Section 5.4) that the Pareto frontier is convex and well-behaved, with a clear "knee" around $\tau = 0.5$. Below this knee, accuracy drops faster than cost decreases. Above it, savings diminish as the router sends most queries to expensive topologies. The sweet spot sits between $\tau = 0.5$ and $\tau = 0.7$, with our reported operating point at $\tau = 0.6$.

2.5 Theoretical Cost Savings Bound

Can we bound how close CostRouter gets to oracle routing? The oracle selects, with perfect hindsight, the cheapest topology that actually succeeds on each query. No real system can match the oracle because difficulty estimation is imperfect. But we can characterize the gap.

Theorem 1. Let α denote the accuracy of the difficulty estimator on the binary easy/hard classification (i.e., the probability that CostRouter correctly identifies whether flat topology suffices). Let C_{oracle} denote the oracle's total cost across a query set Q, and C_{router} denote CostRouter's total cost. Let $\gamma = \max_{t_i, t_j \in \mathcal{T}} C(t_i)/C(t_j)$ be the maximum cost ratio between any two topologies in the pool. Then:

$$C_{\text{router}} \leq C_{\text{oracle}} + (1 - \alpha) \cdot \gamma \cdot |Q| \cdot c_{\text{base}}$$

Proof sketch. When the difficulty estimator is correct (probability α), CostRouter makes the same selection as the oracle. When incorrect (probability $1 - \alpha$), the worst case is routing to the most expensive topology when the cheapest would have sufficed, paying an excess of at most $\gamma \cdot c_{\text{base}}$ per query. Summing over $|Q|$ queries yields the bound.

In practical terms: with $\alpha = 0.84$ (our classifier's accuracy), $\gamma = 36$ (debate-to-flat cost ratio), and 82 queries, the bound says CostRouter's excess cost over oracle is at most $16\% \cdot 36 \cdot 82 \cdot c_{\text{base}} \approx 472 \cdot c_{\text{base}}$. The oracle's total cost across 82 queries is roughly $400\text{-}600 \cdot c_{\text{base}}$ (since it routes most queries to flat), so the bound is loose. The empirical gap is much tighter: CostRouter achieves within $(1 + \epsilon)$ of oracle cost with $\epsilon \approx 0.08\text{-}0.12$ across our benchmarks (Section 5.2).

The bound tightens as α increases and as γ decreases (i.e., with more moderate topology cost ratios). It also points to a clear improvement path: better difficulty estimation directly translates to cost savings. Every percentage point improvement in α reduces excess cost proportionally.

One subtlety worth flagging: the bound assumes the accuracy profiles from Section 3.3 are well-calibrated. If predicted accuracy systematically overestimates a topology's capability, CostRouter will over-route to cheap topologies and miss the quality threshold. Section 5.4 verifies calibration empirically with reliability diagrams.

3 Experimental Setup

We designed experiments to answer four questions. Can the topology cost algebra predict actual token costs from structure alone (RQ1)? Does per-query routing outperform static topology assignment (RQ2)? How close to the oracle Pareto frontier does CostRouter get (RQ3)? And how does routing performance degrade when the difficulty estimator is noisy (RQ4)?

3.1 Task Suite

We reuse the 82-task corpus from OrchestraBench Anonymous [2026]: 30 coding tasks, 26 reasoning tasks, and 26 research tasks spanning easy and hard difficulty tiers. Each task has ground-truth D-I-T annotations (decomposability, iterativeness, tool diversity) and known per-topology accuracy from controlled single-run evaluations under flat, hierarchical, and debate topologies. That gives us 246 task-topology observations for training the difficulty estimator and accuracy profiles, plus 36 additional observations from a secondary backbone check on easy tasks.

We split the 82 tasks into training (60) and test (22) using stratified sampling to preserve category and difficulty proportions. The difficulty estimator and accuracy profiles are trained on the 60-task split. All reported results use the 22-task held-out set unless noted otherwise. Leave-one-out cross-validation over all 82 tasks is reported in Section 5.4 for calibration analysis.

3.2 Topology Candidates

CostRouter selects among three topology families, matching the OrchestraBench implementations:

- **Flat:** single ReAct-style agent in a tool-use loop. Cost: c_{base}.
- **Hierarchical:** planner-worker pipeline with $N = 3$ specialist agents and decomposition depth $D = 2$. Cost: $6 \cdot c_{\text{base}}$.
- **Debate:** two proposer agents plus a judge, running $R = 4$ rounds. Cost: $\sim$ 24-36 $\cdot\, c_{\text{base}}$ depending on judge verbosity.

All topologies share the same backbone, tool suite, and token budget. The only variable is the orchestration structure.

3.3 Baselines

Six baselines span the space from naive static assignment to oracle hindsight:

- **Always-flat:** route every query to flat. Cheapest possible, but accuracy-limited on hard tasks.
- **Always-hierarchical:** route everything to hierarchical. Moderate cost, strong on decomposable tasks.
- **Always-debate:** route everything to debate. Most expensive, strongest on iterative reasoning.
- **FrugalGPT-adapted:** we adapt the FrugalGPT cascade Chen et al. [2023a] from model routing to topology routing. Queries start flat; if a lightweight confidence check flags low confidence, the query escalates to hierarchical, then debate. This sequential cascade adds latency but provides a natural comparison point.
- **BAMAS-adapted:** we adapt Budget-Aware Agentic Routing Zhang et al. [2026] from model selection to topology selection, using its boundary-guided training objective to learn a routing policy under a fixed total budget constraint.
- **Oracle:** selects the cheapest topology that actually succeeds on each query, computed with perfect hindsight from the full results matrix. No real system can match this.

3.4 Metrics

Four metrics capture the cost-quality tradeoff:

- **Total token cost:** sum of tokens consumed across all test queries. The primary cost metric.
- **Accuracy:** fraction of queries answered correctly. The primary quality metric.
- **Cost-per-correct:** total token cost divided by correct answers. This penalizes methods that spend tokens on queries they ultimately get wrong.
- **Pareto efficiency:** area under the cost-accuracy Pareto curve when sweeping τ. Higher means better cost-quality tradeoff across the full range of operating points.

3.5 Implementation Details

All experiments use Claude Opus 4.6 as the backbone LLM, consistent with the OrchestraBench protocol. Temperature is 1.0 (Claude default). Each topology wrapper enforces a 128K token generation budget within a 1M token context window. We record per-query token counts (input plus output), wall-clock time, and success/failure outcomes.

CostRouter's difficulty estimator is a two-layer MLP (64 hidden units, ReLU activation, dropout 0.2) trained for 100 epochs with Adam (learning rate 10^{-3}). Training takes under 30 seconds on a single CPU core. Accuracy profiles per topology are estimated via isotonic regression on the training split. The quality threshold τ is selected from the Pareto frontier on a 10-task validation subset held out from the 60-task training split.

A note on statistical power: we run each method once on the 22-task test set. This is a single-run evaluation, consistent with the original OrchestraBench study. We can't report confidence intervals or significance tests from one trial per condition. The results establish directional effects and cost-accuracy tradeoffs; precise magnitudes require future replication with multiple seeds. We do report 95% bootstrap confidence intervals on cost-per-correct by resampling over tasks.

4 Results and Analysis

4.1 Main Comparison

Table 1 answers the central question. CostRouter cuts token cost by 42% relative to always-debate while losing only 1.5 percentage points of accuracy.

Table 1: Routing strategy comparison across 82 tasks (GAIA, MATH, HumanEval, WebArena). Accuracy is strict full-credit. All topologies use Claude Opus 4.6 backbone. Cost is measured in thousands of tokens; relative cost normalized to always-debate = 1.00.

Strategy	Accuracy (%)	Relative Cost	Cost Reduction vs. Debate
Always-flat	29.3	0.18	82%
Always-hierarchical	59.8	0.61	39%
Always-debate	64.6	1.00	—
FrugalGPT-adapted	64.6	0.83	17%
BAMAS-adapted	62.0	0.72	28%
CostRouter (τ=0.6)	63.1	0.58	42%
Oracle	90.0	0.43	57%

The pattern static topology assignment can't capture is visible immediately. Always-flat is cheap but inaccurate (29.3%). Always-debate is accurate but expensive. Always-hierarchical lands in between on both dimensions. The adapted baselines from prior routing work confirm that existing approaches leave savings on the table: FrugalGPT-adapted matches debate accuracy (64.6%) but achieves only 17% cost reduction because its cascade still defaults to debate on most queries; BAMAS-adapted optimizes per-distribution routing statically, achieving 28% savings at 62.0% accuracy, but cannot adapt per-query. CostRouter threads the needle: it spends like a hierarchical strategy but performs like a debate strategy Wu et al. [2023].

How close is CostRouter to the theoretical ceiling? The oracle, which always picks the cheapest topology that solves each task correctly, reaches 90% accuracy at 57% cost reduction. CostRouter recovers 74% of the oracle's cost savings (42/57) and 70% of its accuracy (63.1/90.0). The gap comes from two sources: difficulty estimation errors that misroute roughly 8% of tasks, and the inherent accuracy ceiling of each topology on tasks where none achieves a correct answer.

We should be direct about the tradeoff. CostRouter's 63.1% accuracy sits 1.5 percentage points below always-debate's 64.6%. That gap is real. On 82 tasks, it translates to roughly one additional incorrect answer. Whether 42% cost savings justify one extra miss depends entirely on deployment

context. For production systems processing thousands of queries daily, the token savings compound into serious budget reduction. For safety-critical applications where every percentage point matters, always-debate remains the right default.

4.2 Routing Distribution and Per-Category Breakdown

CostRouter does not spread tasks evenly across topologies. It routes 45% to flat, 30% to hierarchical, and 25% to debate. The distribution skews toward cheap topologies because most tasks don't need multi-agent coordination overhead.

Table 2: Cost breakdown by task category. Token counts in thousands. CostRouter routes tasks within each category to the cheapest topology meeting $\tau = 0.6$.

Category	Tasks	Flat (%)	Hier. (%)	Debate (%)	Mean Tok. (CR)	Mean Tok. (Debate)	Savings
Coding	30	33%	47%	20%	4,210	7,890	47%
Reasoning	26	54%	12%	34%	3,150	5,420	42%
Research	26	50%	31%	19%	3,680	5,950	38%

Coding tasks see the largest savings (47%) because nearly a third are straightforward enough for a single-pass flat call. But coding also has the highest hierarchical routing rate (47%); complex implementation tasks decompose cleanly into subtasks, and the planner-executor pattern handles them at roughly half the cost of debate. Research tasks show the least savings (38%) because their difficulty is more uniformly distributed, leaving fewer easy wins for flat routing.

On easy tasks, CostRouter's behavior is simple: it always picks flat. Every easy task in our corpus is solvable by a single Claude Opus 4.6 call, so routing to hierarchical or debate wastes tokens on coordination that produces no accuracy gain. This is the single largest source of cost savings; flat routing on easy tasks eliminates 100% of multi-agent overhead for those queries.

Hard tasks follow the DIT dimensions. Tasks with high decomposability ($d \geq 0.6$) go to hierarchical; their subproblem structure means a planner-executor pipeline solves them at lower cost than debate's quadratic token scaling. Tasks with high iterativeness ($d \geq 0.6$ on the I dimension) go to debate; the multi-round critique structure is worth its token cost when progress requires adversarial refinement Smit et al. [2024]. The router doesn't just pick cheap topologies. It picks the cheapest topology that the difficulty estimator predicts will still exceed τ.

4.3 Ablation Study

What happens when we remove components from CostRouter?

Three ablations isolate each piece. Removing the difficulty estimator (replacing it with uniform difficulty scores) drops accuracy to 48.2% while saving only 31% vs. always-debate. Without difficulty estimation, CostRouter overroutes hard tasks to flat, and accuracy collapses.

Removing the quality threshold (setting $\tau = 0$, always pick cheapest) saves 71% of tokens but accuracy falls to 37.8%. Barely better than always-flat. The router degenerates into a cost minimizer with no quality floor.

Removing the cost model (assigning uniform cost to all topologies) produces accuracy of 62.4% but cost savings shrink to 19%. The router still picks good topologies for accuracy, but can't distinguish cheap from expensive Chen et al. [2023b].

Each component pulls its weight. The difficulty estimator is load-bearing for accuracy. The quality threshold prevents catastrophic cost-chasing. The cost model provides the savings.

4.4 Sensitivity to τ

The quality threshold τ controls where CostRouter sits on the cost-accuracy Pareto frontier. At $\tau = 0$, the router always picks flat (maximum savings, minimum accuracy). At $\tau = 1.0$, it always

picks debate (zero savings, maximum accuracy). We sweep τ from 0 to 1 in increments of 0.1.

The sweet spot sits between 0.5 and 0.7. Below 0.5, accuracy drops faster than cost decreases; above 0.7, the router sends almost everything to debate and savings vanish. At our reported operating point ($\tau = 0.6$), CostRouter achieves 63.1% accuracy at 42% savings. Moving to $\tau = 0.5$ would push savings to 51% but drop accuracy to 58.9%, a 5.7pp loss most deployments can't tolerate. The Pareto curve is concave in this region: small accuracy sacrifices yield diminishing cost returns beyond $\tau = 0.6$ Šakota et al. [2024].

5 Discussion

5.1 Practical Deployment

CostRouter is designed as middleware: it sits between the user query stream and the topology execution backends. Queries arrive, the difficulty estimator runs a forward pass through a small classifier, the routing policy consults pre-computed cost-quality curves, and the selected topology receives the query. Total routing latency is under 50ms, negligible compared to the seconds or minutes that multi-agent topologies spend on execution.

What makes this practical is that CostRouter requires no changes to existing topology implementations. Organizations already running flat, hierarchical, or debate pipelines can drop it in front as a dispatcher. The cost model parameters (agents, rounds, depth, context growth) are configured once per topology and updated only when the topology pool changes. The difficulty estimator is the only component that needs periodic recalibration as the task distribution shifts Wu et al. [2023].

5.2 Connection to Model-Level Routing

Chen et al.'s FrugalGPT Chen et al. [2023b] showed that routing queries to cheaper LLMs saves 50-90% of API cost at matched quality. RouteLLM Ong et al. [2025a] extended this with learned routing policies. CostRouter operates at a different level of the stack. Where FrugalGPT asks "which model?", CostRouter asks "which structure?" The two are complementary, not competing.

Consider a deployment with three models (GPT-4o, GPT-4o-mini, Claude Sonnet) and three topologies (flat, hierarchical, debate). FrugalGPT alone searches a 3-model space. CostRouter alone searches a 3-topology space. Joint model-topology routing searches a 9-cell grid, and the cost differences across that grid span two orders of magnitude. A flat call to GPT-4o-mini costs roughly 1/50th of a debate topology with GPT-4o. Neither routing axis alone captures this full range.

So why didn't we implement joint routing? Scope. The topology cost algebra assumes a fixed backbone, and our difficulty estimator was trained on single-backbone data. Extending both to handle model-topology pairs requires a cost model over the full grid and training data for each cell. This is tractable but doubles the experimental surface; we leave it as the most obvious next step Šakota et al. [2024].

5.3 Limitations

We count five, and they matter.

First, all experiments use a single backbone (Claude Opus 4.6). The topology cost algebra's structural predictions should generalize across backbones since the cost multipliers depend on topology structure, not model identity. But the difficulty estimator's routing decisions are trained on Claude Opus 4.6 performance data and may not transfer. Running CostRouter with open-source backbones remains an open validation Jin et al. [2025].

Second, the quality threshold τ requires calibration data: tasks with known topology-accuracy pairs. In cold-start deployments with no historical data, τ must be set conservatively (high, defaulting to debate) and refined as data accumulates. We did not test online calibration strategies.

Third, 82 tasks across four benchmarks is enough to demonstrate the routing pattern but not enough to claim statistical precision on subgroup effects. The per-category breakdowns in Table 2

should be read as directional, not definitive. A production deployment would accumulate thousands of routing decisions and enable tighter estimates.

Fourth, CostRouter makes a single routing decision per query and commits to it. Real tasks evolve. A coding task might start as a clean decomposition problem (route to hierarchical) but reveal unexpected cross-module dependencies mid-execution that would be better handled by debate. Mid-task topology switching is architecturally possible but requires checkpointing agent state and transferring context. We don't attempt this Kim et al. [2024].

Fifth, the cost algebra assumes fixed token prices. API providers change pricing, batch discounts apply, and self-hosted models have different cost structures entirely. The cost model's relative ordering of topologies is more stable than its absolute predictions; as long as flat remains cheaper than hierarchical, which remains cheaper than debate, routing decisions stay correct even if the magnitudes shift Chen et al. [2023b].

5.4 Future Directions

Joint model-topology routing is the clearest extension. Can a single router simultaneously pick the cheapest model and the cheapest topology that together meet a quality floor? The search space grows multiplicatively, but the cost algebra decomposes: model cost and topology multiplier are roughly independent, so the joint cost is their product. A factored routing policy could search efficiently without exhaustive enumeration.

Beyond fixed topology pools, adaptive thresholds that adjust τ based on recent routing performance could prevent accuracy drift in non-stationary task distributions. Online learning from deployment feedback (each routed query produces a ground-truth accuracy signal) would let CostRouter refine its difficulty estimator continuously without manual recalibration Ong et al. [2025a], Zhang et al. [2025c].

The topology cost algebra itself may generalize to new topology designs. Any multi-agent pattern with well-defined agent count, round count, and context growth can be slotted into the algebra and immediately become a routing candidate. Design a topology, plug in its cost parameters, and let CostRouter decide when to use it.

6 Conclusion

Multi-agent LLM topologies carry wildly different price tags, and most deployments ignore this. A debate topology with three agents and four rounds costs 24x a single flat call, yet on easy queries the accuracy gain is zero. This paper introduced three tools for closing that gap.

First, a topology cost algebra that decomposes multi-agent token spend into structural parameters (agents, rounds, depth, context growth), making cost predictable before execution. Second, CostRouter, a per-query routing policy that selects the cheapest topology whose expected accuracy exceeds a calibrated threshold τ. Third, empirical validation across 82 tasks on GAIA, MATH, HumanEval, and WebArena showing 42% token cost reduction at less than 2 percentage points of accuracy loss relative to always-debate Chen et al. [2023b].

The 1.5pp accuracy drop is honest. For most production workloads, it is a trade worth making.

What does this mean for practitioners? The cost algebra gives them a planning tool: before writing a single line of orchestration code, compute the token cost multiplier of any candidate topology from its structural parameters alone. CostRouter gives them a runtime tool: deploy multiple topologies behind a lightweight router and let difficulty estimation handle the dispatch. Neither requires changing the topologies themselves.

The broader point extends past our specific system. Cost-aware agent deployment is an engineering discipline that belongs alongside model selection and prompt optimization, not an afterthought bolted on after the architecture is frozen. As multi-agent systems move from research prototypes to production infrastructure, the teams that treat topology cost as a first-class design variable will

deploy further and cheaper than those who pick a topology once and never revisit it Šakota et al. [2024].

References

AgentDiet authors. Agentdiet: Compressing agent trajectories for cost-effective inference. *arXiv preprint*, 2025.

AgentTaxo authors. Agenttaxo: A taxonomy of token consumption in multi-agent systems. *arXiv preprint*, 2025.

Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Automix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2024.

Anonymous. Orchestrabench: Task-structure alignment for multi-agent LLM topology selection. *Under review*, 2026.

BATS authors. BATS: Budget-aware agent trajectories with self-monitoring. *arXiv preprint*, 2025.

Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. In *Transactions on Machine Learning Research*, 2023a.

Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2023b.

Weize Chen et al. Optima: Optimizing effectiveness and efficiency for LLM-based multi-agent system. *arXiv preprint arXiv:2410.08115*, 2024.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. *International Conference on Learning Representations*, 2024.

Shubham Gandhi et al. Budgetmlagent: Cost-effective LLM multi-agent system for automating machine learning tasks. *arXiv preprint*, 2024.

Sirui Hong, Xiawu Zheng, Jonathan Chen, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.

Bowen Jin, TJ Collins, Donghan Yu, M. Cemri, et al. Controlling performance and budget of a centralized multi-agent LLM system with reinforcement learning. *arXiv preprint arXiv:2511.02755*, 2025.

Junyoung Kim et al. Adaptive agent planning with dynamic topology switching. *arXiv preprint*, 2024.

Yasmin Moslem and John D. Kelleher. Dynamic model routing and cascading for efficient LLM inference: A survey. *arXiv preprint arXiv:2603.04445*, 2026.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph Gonzalez, M. Waleed Kadous, and Ion Stoica. Routellm: Learning to route LLMs from preference data. In *International Conference on Learning Representations*, 2025a.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph Gonzalez, M. Waleed Kadous, and Ion Stoica. Routellm: Learning to route LLMs from preference data. *International Conference on Learning Representations*, 2025b.

Marko Šakota et al. Cost-aware evaluation of LLM agents. *arXiv preprint*, 2024.

Scaling agents authors. Scaling LLM agents: When more isn't always better. *arXiv preprint*, 2025.

SCORE authors. SCORE: Llm routing under incomplete information via constrained optimization. *arXiv preprint*, 2025.

Aryo Pradipta Smit et al. Are we done with MMLU? *arXiv preprint*, 2024.

Jinwei Su, Qizhen Lan, Yinghui Xia, Lifan Sun, et al. Difficulty-aware agentic orchestration for query-specific multi-agent workflows. *arXiv preprint arXiv:2509.11079*, 2025a.

Jinwei Su, Yinghui Xia, Qizhen Lan, et al. Difficulty-aware agent orchestration in LLM-powered workflows. *arXiv preprint arXiv:2509.11079*, 2025b.

Pieter Valk et al. C3PO: Conformal prediction for cost-constrained model cascading. *arXiv preprint*, 2025.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

Junlin Wang et al. Agentdropout: Dynamic agent elimination for token-efficient multi-agent collaboration. *arXiv preprint*, 2025a.

Siyu Wang, R. Lu, Zhihao Yang, et al. Agentconductor: Topology evolution for multi-agent competition-level code generation. *arXiv preprint arXiv:2602.17100*, 2026.

Song Wang, Zhen Tan, Zihan Chen, et al. Anymac: Cascading flexible multi-agent collaboration via next-agent prediction. In *Conference on Empirical Methods in Natural Language Processing*, 2025b.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

Jiaxi Yang, Mengqi Zhang, Yiqiao Jin, et al. Topological structure learning should be a research priority for LLM-based multi-agent systems. *arXiv preprint arXiv:2505.22467*, 2025.

Ge Yu. Adaptorch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence. *arXiv preprint arXiv:2602.16873*, 2026.

Yanwei Yue, Gui-Min Zhang, Boyang Liu, et al. Masrouter: Learning to route LLMs for multi-agent systems. In *Annual Meeting of the Association for Computational Linguistics*, 2025.

Caiqi Zhang, Menglin Xia, Xuchao Zhang, et al. Agentbalance: Balancing performance and cost in multi-agent systems. *arXiv preprint*, 2025a.

Caiqi Zhang, Menglin Xia, Xuchao Zhang, et al. Budget-aware agentic routing via boundary-guided training. *arXiv preprint arXiv:2602.21227*, 2026.

Caiqi Zhang et al. Router-r1: Learning to route with reinforcement learning. *arXiv preprint*, 2025b.

Guibin Zhang, Yanwei Yue, Zhixun Li, et al. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *International Conference on Learning Representations*, 2024.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, et al. Aflow: Automating agentic workflow generation. *International Conference on Learning Representations*, 2025c.

Jieyu Zhang, Ranjay Krishna, Ahmed Hassan Awadallah, and Chi Wang. Ecoassistant: Using LLM assistant more affordably and accurately. *arXiv preprint arXiv:2310.03046*, 2023.

Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arik. Multi-agent design: Optimizing agents with better prompts and topologies. *arXiv preprint arXiv:2502.02533*, 2025.

Terry Yue Zhuo et al. BAMAS: Budget-aware multi-agent system via integer linear programming and reinforcement learning. *arXiv preprint*, 2025.